Operating Systems — Assignment 7 Report

xv6: Threads, Semaphores, Deadlock Detection, Priority Scheduling

Source files are in `src/`, terminal screenshots in `ss/`. Tested on xv6 (x86) under QEMU (2 CPUs for Q1–Q3, 1 CPU for Q4).

Question 1 — Threads (`clone` / `join`)

Design. `clone(fcn, arg, stack)` calls `allocproc()` like `fork()`, but instead of `copyuvm()` it sets `np->pgdir = curproc->pgdir`, so the whole address space is shared. The thread keeps its own pid, kernel stack and trap frame. The kernel writes `arg` and a fake return address on the user stack (`stack` is the lowest address of a `PGSIZE` region), then sets `tf->eip = fcn` and `tf->esp`. `struct proc` gained two fields: `isthread` and `leader` (the process that owns the page directory; threads created by threads point to the same leader).

exit / wait / join / fork.

- `wait()` skips thread children, and a process is only reaped once no live thread of its group remains (zombie threads are reaped first), so a shared `pgdir` is never freed under a running thread.
- `join()` reaps only threads: it frees the kernel stack and slot and **never calls `freevm()`**.
- `exit()` of an ordinary process marks its remaining threads killed; a thread's `exit()` also wakes its leader's parent, which may be waiting for the group to disappear. Orphaned threads are never handed to `init`.
- `fork()` is unchanged (child is an ordinary process with a copied address space).

Result (`threadtest`, 3 threads × 100 000 increments, no locking):

run	1	2	3	4	5
final counter	168 205	167 765	172 767	169 451	166 007

Always below 300 000: the threads share one `counter`, and a thread that was interrupted (or is running on another CPU) between its load and its store overwrites the updates made in the meantime.

Problems and fixes.

1. *No race appeared at first.* GCC emitted a single `addl` to memory, so the increment was rewritten as an explicit load/add/store on a `volatile`; and the window between load and store is only a few instructions, so a short delay loop was added between them. With the wider window the race shows on one CPU (timer preemption between load and store) as well as on two.
2. *Leader exits while threads still run* (tested with spinning threads): handled by killing the threads in `exit()` and holding back `freevm()` in `wait()`; without that the shell could free live memory.
3. *Limitation:* `sz` is copied at clone time, so heap growth after cloning is invisible to existing threads.

Question 2 — Semaphores

Design. `sem.c` holds a table of 10 `struct semaphore { value; wakeups; inuse; lk }`. A table spinlock protects initialisation; each semaphore has its own spinlock. `sem_wait` does `value--` and blocks if `value < 0`; `sem_signal` does `value++` and wakes a waiter if `value <= 0`, as specified.

xv6's `wakeup(chan)` wakes *all* sleepers, which would let every waiter through on one signal. Fix: a `wakeups` ticket counter — `sem_signal` adds one ticket, and a woken waiter only proceeds if it can take one, otherwise it sleeps again. A waiter killed while blocked undoes its decrement and returns −1.

Before / after the race (`synctest` vs `threadtest`):

program	synchronisation	final counter (expected 300 000)
`threadtest`	none	≈ 166 000 – 173 000 (race)
`synctest` part 1	`sem_wait` / `sem_signal` around each increment	**300 000** (exact)

Producer-Consumer (buffer size 5, 15 items, semaphores `empty`=5, `full`=0, `mutex`=1, producer and consumer as two threads). The consumer starts late, so the log shows the buffer filling to 5, the producer blocking, then the two alternating; at the end: 15 items consumed in FIFO order, max fill 5, min fill 0, buffer empty — no overflow or underflow (`ss/q2_synctest.png`).

Question 3 — Deadlock detection and avoidance (`deadlock.c`)

Data structures. Static arrays `Allocation[n][m]`, `Request[n][m]`, `Max[n][m]`, `Available[m]`; `Need = Max − Allocation`.

Functions. `detect_deadlock()`: `Work = Available`; processes holding nothing start finished; repeatedly finish any process with `Request <= Work` (`Work += Allocation`); the unfinished ones are returned in `deadlocked[]`. `is_safe_state()`: the safety algorithm on `Need`. `request_resources()`: check `req <= Need`, `req <= Available`, pretend-allocate, run `is_safe_state()`, roll back if unsafe. State (Available / Allocation / Request-or-Need) and every step's `Work` vector are printed.

Results (all 8 built-in checks pass):

test	matrix	outcome
1	5 processes × 3 resources, `Available = <0 0 0>`	**DEADLOCK: P1 P2 P3 P4** (only P0 can finish)
2	same allocation, P2 no longer requests	no deadlock (order P0, P2, P3, P4, P1)
3a	Banker's, `Available = <3 3 2>`	safe, sequence P1, P3, P4, P0, P2
3b	P1 requests `<1 0 2>`	**granted** (still safe)
3c	P4 requests `<3 3 0>`	must wait (more than Available)

3d	P0 requests `<0 2 0>`	**denied: unsafe**, state rolled back
pipes	real processes A, B	**DEADLOCK: P0 P1** (circular wait)

Real deadlock. A keeps pipe 1's write end and blocks reading pipe 2; B keeps pipe 2's write end and blocks reading pipe 1. Each `read()` can only return when data arrives or all write ends close, and each write end is held by the other process. A monitor process closes all its pipe ends, collects each child's "I hold X, I want Y" report, builds the matrices, and `detect_deadlock()` reports the cycle. Running `deadlock pause` and pressing Ctrl-P shows both processes in state `sleep` inside `piperead` — direct evidence that they are blocked. The monitor then kills both children.

Problem. Killed children exit immediately after `read()` returns −1 (xv6 checks `killed` on syscall return), so they cannot print a message afterwards; the monitor reports on their behalf.

Question 4 — Priority scheduling with aging

Design. `struct proc` gained `priority` (1 = highest … 20 = lowest, default 10 set in `allocproc`, not inherited) and `waitticks`. `scheduler()` scans the table circularly from the slot after the last one run and keeps the RUNNABLE process with the numerically lowest priority; only a strictly better priority replaces the candidate, so equal priorities rotate round-robin. `agetick()` runs once per timer tick (from `trap.c` on CPU 0): each RUNNABLE process's `waitticks` is incremented, and above 30 its priority number drops by 1 (never below 1) and the counter resets; the counter is zeroed when a process is scheduled or stops being RUNNABLE. `set_priority(pid, p)` (valid 1–20) and `get_priority(pid)` are system calls; an extra `get_waitticks(pid)` lets `priotest` show the aging. Preemption happens at tick granularity (xv6 yields on each tick).

Result (`priotest`, 1 CPU, 4 CPU-bound children of 200 M iterations):

completion #	pid	start priority	final priority (after aging)	finished at tick
1	4	2	2	62
2	5	6	5	129
3	6	12	8	191
4	7	**20**	14	254

Completion order equals priority order. The parent's samples (`ss/q4_priotest.png`) show the waiting processes' priority number falling by one about every 31 ticks (pid 7: 20 → 19 → 18 → 17 → 16 → 15 → 14) and `wait` resetting after each boost — aging at work. The priority-20 process completes; without aging it would sit behind every other process indefinitely.

Problems and fixes.

1. Each child originally called `set_priority()` on itself when first scheduled, which silently *undid* the boost it had earned while waiting. Removed — the parent sets priorities right after `fork`.
2. The parent observer runs at priority 1 so that it can wake from `sleep()` and sample while children compute.
3. On 2 CPUs the two best processes run simultaneously, blurring the order — hence 1 CPU for this experiment.

Screenshots (folder `ss/`)

file	shows
`q1_threadtest.png`	`threadtest`: final counter below 300 000
`q2_synctest.png`	`synctest`: counter exactly 300 000 and the producer-consumer log
`q3_deadlock.png`	`deadlock`: detection, Banker's tests and the real pipe deadlock
`q3_deadlock_ctrlp.png`	`deadlock pause` + Ctrl-P: both processes asleep in `piperead`
`q4_priotest.png`	`priotest` (1 CPU): priority/wait samples and completion order